Android
Unidad 4. Arquitectura

PROGRAMACIÓN MULTIMEDIA Y DE DISPOSITIVOS MÓVILES

2º DAM

Índice

1. Ciclo de vida de una Actividad
2. Arquitectura de la aplicación
3. Ejemplo

1. Ciclo de vida de una Actividad

Por ahora estamos asumiendo que el usuario siempre estará en nuestra aplicación. Pero qué pasa si el usuario decide hacer una llamada o buscar algo por Internet, es decir, abandonar la aplicación y hacer otra cosa.

Si los usuarios van a otras aplicaciones y no vuelven a la aplicación, *Android* finalmente destruirá nuestra *activity* (la ejecución de la aplicación).

Así utiliza esos recursos para ejecutar las aplicaciones más recientes con las que el usuario interactúa.

Como desarrolladores tenemos que estar listos para cuando el usuario abandone nuestra aplicación en cualquier momento o para que el sistema destruya nuestra *activity* porque necesite esos recursos para otras.

Un ejemplo de esto son los **borradores** de los correos, que aunque no los usemos en un tiempo, siempre estarán listos para volver a ellos y continuar escribiendo el mensaje. No se borra nada.

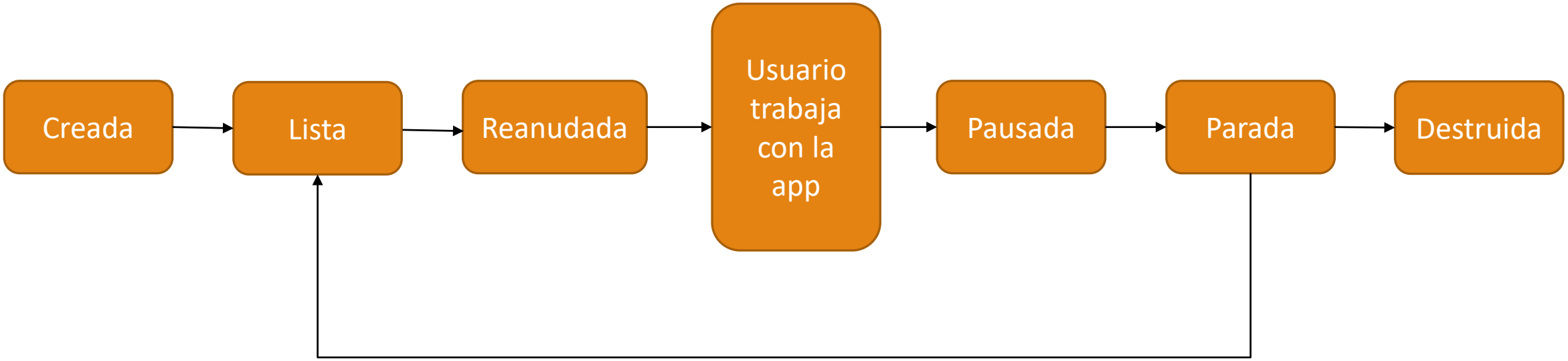
1. Ciclo de vida de una Actividad

La razón de todo esto es por la conservación de la memoria. *Android* quiere tener suficientes recursos para la aplicación actual, así la interacción del usuario con ella es rápida, *responsive* (adaptativa) y los recursos no se malgastan con otras aplicaciones que el usuario no ha interactuado por un tiempo.

Android tiene un ciclo de vida (**lifecycle**) que las **Actividades** tienen que seguir. El sistema pasará de un estado a otro y ofrecen *callbacks* con los que nosotros, como desarrolladores, seremos notificados cuando esos cambios pasen en nuestra Actividad.

Cuando seamos notificados tendremos la oportunidad de ejecutar ciertas instrucciones que necesitemos.

1. Ciclo de vida de una Actividad



1. Ciclo de vida de una Actividad

Cuando una Actividad es lanzada, se crea (**created**), pasa a **started**, y después a **resumed**. En este punto la Actividad está activa y el usuario puede interactuar con ella.

Tan pronto como el usuario se vaya a otra actividad o aplicación diferente, esta Actividad pasa a **paused**, y después al estado **stopped**. Si es cerrada pasa a **destroyed**.

Si el usuario vuelve atrás y la reinicia volvemos al estado **started** y el usuario puede interactuar con ella otra vez.

Este es un pequeño resumen del ciclo de vida de un Activity.

En la documentación podemos ver el diagrama completo:

https://developer.android.com/guide/components/activities/activity-lifecycle

1. Ciclo de vida: Actividad

Una vez visto el ciclo de vida de una actividad: ¿Cómo le puedo decir al sistema que me avise sobre ciertas transiciones? Como por ejemplo, solo quiero mostrar una animación en la pantalla si el usuario está mirando activamente la actividad y parar la animación cuando esa pantalla ya no sea visible.

El sistema te avisará que una transición en el ciclo de vida está pasando a través de un método *callback*.

Todas las transiciones tienen sus propios *callbacks*, así que usándolos nos da una oportunidad, como desarrolladores, para intervenir y ejecutar operaciones, tales como restaurar o salvar algún dato o limpiar referencias que ya no necesitas.

Si nos fijamos, hasta ahora, todo el código de nuestras actividades ha sido introducido en el método **onCreate**.

En la documentación podemos ver todos los métodos *callback* para los estados del ciclo de vida que podemos sobreescribir e insertar código:

https://developer.android.com/guide/components/activities.html?hl=ES

1. Ciclo de vida de una Actividad

Ejemplo:

MainActivity.kt

```kotlin
package com.example.ud4_ejemplo1

import ...

// Etiqueta para el Log
private const val TAG = "Ud4_Ejemplo1"

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Ud4_Ejemplo1Theme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
```

```kotlin
    override fun onStart() {
        super.onStart()
        Log.d(TAG, msg: "onStart")
    }

    override fun onResume() {
        super.onResume()
        Log.d(TAG, msg: "onResume")
    }

    override fun onRestart() {
        super.onRestart()
        Log.d(TAG, msg: "onRestart")
    }

    override fun onPause() {
        super.onPause()
        Log.d(TAG, msg: "onPause")
    }

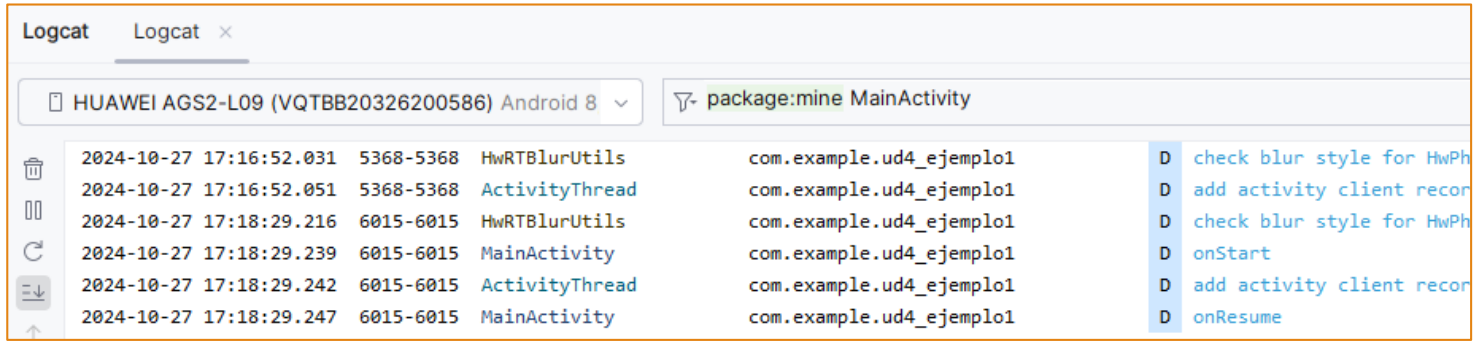
    override fun onStop() {
        super.onStop()
        Log.d(TAG, msg: "onStop")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, msg: "onDestroy")
    }
}
```

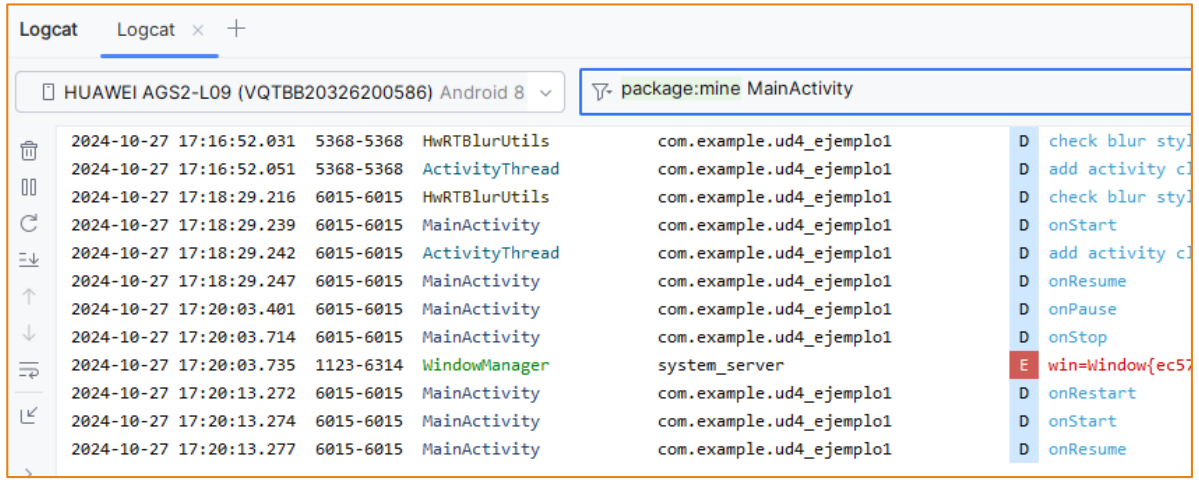
1. Ciclo de vida: Actividad

Ejemplo:

Al iniciar la aplicación:



Al salir y volver a entrar en la aplicación:



2. Arquitectura de la aplicación

Una vez llegados a este punto, y antes de avanzar, es importante definir la arquitectura de nuestra aplicación que le permita crecer, aumentar su solidez y facilitar su prueba.

Algunos de los principios arquitectónicos más comunes son:

- **La separación de problemas**: indica que la app debe dividirse en clases de funciones, cada una con responsabilidades independientes.
- **El control de la IU a partir de un modelo**: establece que debes controlar tu IU a partir de un modelo, preferentemente un modelo persistente. Los modelos son componentes responsables de administrar los datos de una app. Son independientes de los componentes de la app y los elementos de la IU, de modo que no se ven afectados por el ciclo de vida de la app ni los problemas asociados.

2. Arquitectura de la aplicación

Por lo tanto cada aplicación debería tener al menos dos capas:

- Capa de la IU: Es una capa que muestra los datos en la pantalla, totalmente independiente de los datos.
- Capa de datos: Es una capa que almacena, recupera y expone los datos.

Además, puede haber una capa más, la capa de dominio, para simplificar y volver a utilizar las interacciones entre las capas de datos y de la IU.

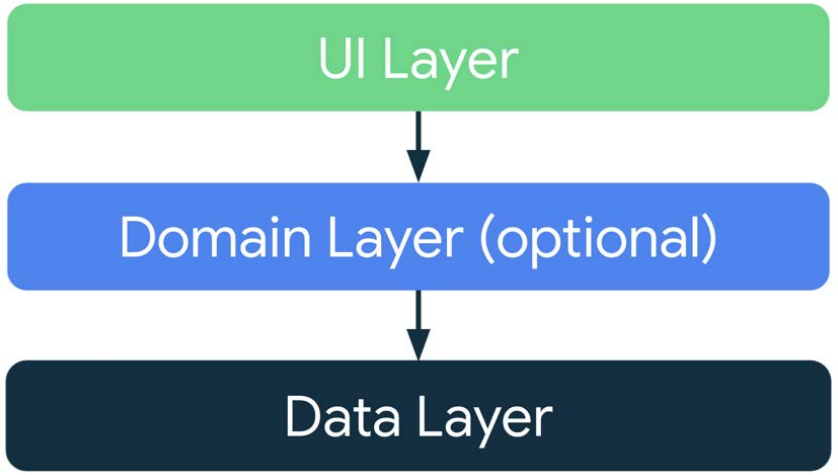


Imagen de developer.android.com

2. Arquitectura de la aplicación

La función de la capa de la IU consiste en mostrar los datos de la aplicación en la pantalla. Cuando los datos cambian debido a una interacción del usuario, como cuando se presiona un botón, la IU debe actualizarse para mostrar los cambios.

La **capa de la IU** consta de los siguientes componentes:

- **Elementos de la IU**: Son los componentes que renderizan los datos en la pantalla. Estos elementos se compilan con *Jetpack Compose*.
- **Contenedores de estado**: Son los componentes que contienen los datos, los exponen a la IU y controlan la lógica de la app. Un ejemplo de contenedor de estado es ***ViewModel***.

2. Arquitectura de la aplicación

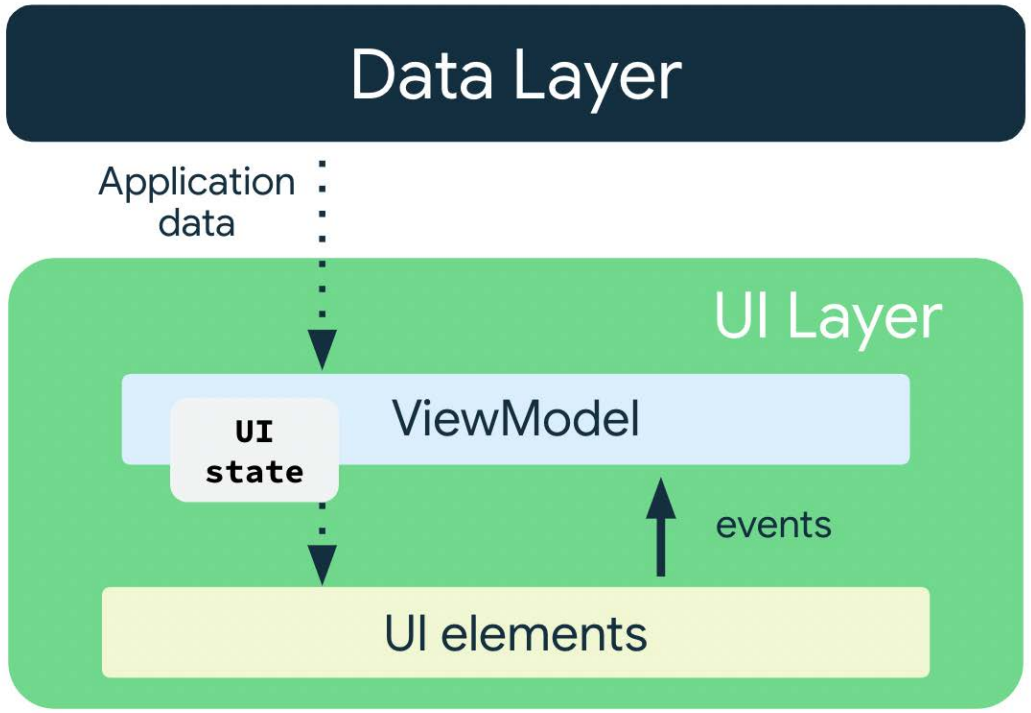


Imagen de developer.android.com

El flujo es el siguiente:

- ViewModel conserva y expone el estado que consumirá la IU.
- El estado de la IU son los datos de aplicación que transforma ViewModel.
- La IU notifica al ViewModel los eventos de usuario.
- ViewModel controla las acciones del usuario y actualiza el estado.
- El estado actualizado se envía a la IU para su renderización.

El proceso se repite para cualquier evento que modifique el estado.

2. Arquitectura de la aplicación

El componente ***ViewModel*** contiene y expone el estado que consume la IU. Ese **estado de la IU** son los datos de la aplicación que transforma *ViewModel*.

ViewModel almacena los datos relacionados con la app que no se destruyen cuando el *framework* de *Android* destruye y vuelve a crear la actividad.

La aplicación retiene automáticamente objetos *ViewModel* durante los cambios de configuración de modo que los datos que tenga estén disponibles de inmediato después de la recomposición. Por ejemplo, al girar el dispositivo.

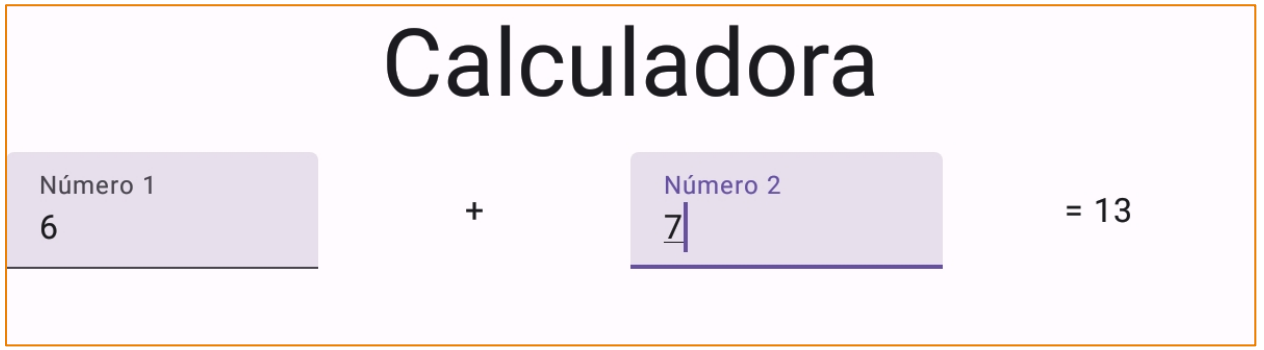
El estado de la IU más sus elementos son lo que forman la IU.



Imagen de developer.android.com

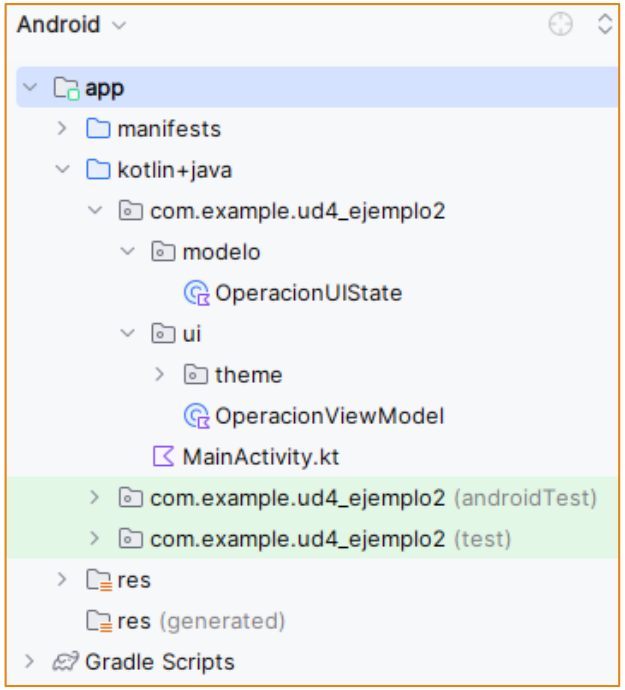
3. Ejemplo

En el siguiente ejemplo vamos a realizar una pequeña calculadora que sume dos números. Tantos los números como el resultado se almacenarán en un UIstate y trabajaremos con ellos mediante ViewModel.



3. Ejemplo

La estructura del proyecto será la siguiente:



3. Ejemplo

Creamos una clase ***OperacionUIState*** dentro del paquete **modelo** para almacenar tanto los números como el resultado.

OperacionUIState.kt

```kotlin
package com.example.ud4_ejemplo2.modelo

data class OperacionUIState(
    val numero1: String = "",
    val numero2: String = "",
    val resultado: String = ""
)
```

3. Ejemplo

Para poder utilizar la clase ***ViewModel*** tenemos que añadir sus dependencias.

libs.versions.toml

```toml
[versions]
agp = "8.4.0"
kotlin = "1.9.0"
coreKtx = "1.13.1"
junit = "4.13.2"
junitVersion = "1.2.1"
espressoCore = "3.6.1"
lifecycleRuntimeKtx = "2.8.6"
activityCompose = "1.9.2"
composeBom = "2024.04.01"
viewmodel = "2.8.6"

[libraries]
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }
androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }
androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
androidx-ui = { group = "androidx.compose.ui", name = "ui" }
androidx-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
androidx-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
androidx-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
androidx-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
androidx-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
androidx-material3 = { group = "androidx.compose.material3", name = "material3" }
androidx-viewmodel = { group = "androidx.lifecycle", name = "lifecycle-viewmodel-compose", version.ref = "viewmodel"}

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
jetbrains-kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }
```

build.gradle.kts (:app)

```
dependencies {

    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation(libs.androidx.activity.compose)
    implementation(platform(libs.androidx.compose.bom))
    implementation(libs.androidx.ui)
    implementation(libs.androidx.ui.graphics)
    implementation(libs.androidx.ui.tooling.preview)
    implementation(libs.androidx.material3)
    implementation(libs.androidx.viewmodel)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(platform(libs.androidx.compose.bom))
    androidTestImplementation(libs.androidx.ui.test.junit4)
    debugImplementation(libs.androidx.ui.tooling)
    debugImplementation(libs.androidx.ui.test.manifest)
}
```

3. Ejemplo

Creamos una clase ***OperacionViewModel*** dentro del paquete **ui** donde controlaremos y trabajaremos con los números y el resultado.

OperacionViewModel.kt

```
1  package com.example.ud4_ejemplo2.ui
2
3  import ...
12
13 class OperacionViewModel : ViewModel() {
14     private val _operacionUItate = MutableStateFlow(OperacionUIState())
15
16     val operacionUIState: StateFlow<OperacionUIState> = _operacionUItate.asStateFlow()
17
18     var respuestaNumero1 by mutableStateOf( value: "")
19         private set
20
21     var respuestaNumero2 by mutableStateOf( value: "")
22         private set
23
```

StateFlow es un flujo observable contenedor de datos que emite actualizaciones de estado actuales y nuevas.
Su propiedad *value* refleja el valor de estado actual.
Para actualizar el estado y enviarlo al flujo, asigna un nuevo valor a la propiedad de la clase *MutableStateFlow*.

_operacionUIState solo es accesible y editable dentro de *OperacionViewModel*.
La IU puede leer su valor usando la propiedad *operacionUIState* de solo lectura.
Con *asStateFlow* conseguimos que este flujo de estado mutable sea de solo lectura.

3. Ejemplo

La función *update* permite actualizar el estado de *_operacionUIState*.
La función *copy* copia un objeto, lo que te permite modificar algunas de sus propiedades sin modificar el resto.

```
24      fun actualizarNumero1(num: String) {
25          respuestaNumero1 = num
26
27          _operacionUItate.update { estadoActual ->
28              estadoActual.copy(
29                  numero1 = respuestaNumero1
30              )
31          }
32
33          calcular()
34
35      }
36
37      fun actualizarNumero2(num: String) {
38          respuestaNumero2 = num
39
40          _operacionUItate.update { estadoActual ->
41              estadoActual.copy (
42                  numero2 = respuestaNumero2
43              )
44          }
45
46          calcular()
47      }
48
49      private fun calcular() {
50          _operacionUItate.update { estadoActual ->
51              val num1 = estadoActual.numero1.toIntOrNull() ?: 0
52              val num2 = estadoActual.numero2.toIntOrNull() ?: 0
53
54              estadoActual.copy(
55                  resultado = (num1 + num2).toString()
56              )
57          }
58      }
59  }
```

3. Ejemplo

MainActivity.kt

```kotlin
package com.example.ud4_ejemplo2

import ...

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Ud4_Ejemplo2Theme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Calculadora(
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}
```

```
@Composable
fun Calculadora(
    modifier: Modifier = Modifier,
    operacionViewModel: OperacionViewModel = viewModel()
    ) {

    // Cada vez que haya un cambio en el valor de operacionUIState se produce una
    // recomposición para los elementos componibles con el valor de uiState.
    val uIState by operacionViewModel.operacionUIState.collectAsState()

    Column(
        verticalArrangement = Arrangement.spacedBy(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = modifier
    ){
        Text(
            text = stringResource("Calculadora"),
            style = MaterialTheme.typography.displayMedium
        )
        Row(
            verticalAlignment = Alignment.CenterVertically
        ){
           TextField(
               // El número a mostrar será el que está almacenado en el uIState
               value = uIState.numero1,
               label = {Text (text = stringResource("Número 1")) },
               onValueChange = {
                   // Al escribir el número lo actualizamos en nuestro ViewModel
                   operacionViewModel.actualizarNumero1(it)
               },
               modifier = Modifier.weight(1F)
                   .onFocusChanged { focus ->
                       if (focus.isFocused) {
                           operacionViewModel.actualizarNumero1( num: "")
                       }
                   }
           )
           Text(
               text = "+",
               textAlign = TextAlign.Center,
               modifier = Modifier.weight(1F)
           )
           TextField(
               // El número a mostrar será el que está almacenado en el uIState
               value = uIState.numero2,
               label = {Text (text = stringResource("Número 2"))},
               onValueChange = {
                   // Al escribir el número lo actualizamos en nuestro ViewModel
                   operacionViewModel.actualizarNumero2(it)
               },
               modifier = Modifier.weight(1F)
                   .onFocusChanged { focus ->
                       if (focus.isFocused) {
                           operacionViewModel.actualizarNumero2( num: "")
                       }
                   }
           )
           Text(
               text = "= ${uIState.resultado}",
               textAlign = TextAlign.Center,
               modifier = Modifier.weight(1F)
           )
       }
   }
}
```